ASSIGNMENT 2
NLP

N-gram Language Models · Sentence Generation · Urdu Fake News Detection

Muhammad Umer Shehzad

Roll No: BS-23-IB-101047

Department: CIS 23-27

Submitted to:

Sir Ijaz Hussain

Due Date: 09-03-2026

Assignment Overview

This report documents the complete implementation of NLP Assignment 2. The work covers four tasks that progressively build expertise in probabilistic language modelling and Naive Bayes text classification, all implemented from scratch in Python within a Jupyter Notebook on collab.

Assignment Scope

Part	Task	Key Techniques
1	N-gram Language Models	Bigrams, Trigrams along with smoothing
2	Sentence Generation	Simple Probability (greedy), Shannon's Method (stochastic)
3	Out-of-Domain Evaluation	Perplexity, Coverage %, OOV Rate, Avg Probability
4	Urdu Fake News Detection	Naive Bayes, Boolean NB, Negation handling, Stop words

Preprocessing Architecture

A dedicated preprocessing module (Cell 9) is defined before any corpus loading. It provides three reusable functions:

- **clean_pdf_text()** Fixes PDF extraction artifacts: resolves ligatures (fi→fi, fl→fl) via NFKC normalization, repairs hyphenated line-breaks (dis-\nambiguation→disambiguation), removes citation markers [1],[2-5], strips page-number lines, and drops short ALL-CAPS header/footer lines.
- **heuristic_segment()** Rule-based sentence segmenter. Splits on sentence-ending punctuation (. ! ?) followed by a capital letter, while suppressing splits on 30+ known abbreviations (et al., fig., Dr., vol., etc.) and single-character initials. Also splits on paragraph double-newlines.
- **urdu_normalize()** Standardises Arabic character variants to Urdu equivalents (ه→ہ, ى→ي, ک→ك, ة→ت), removes diacritics/tashkeel (U+064B–U+065F), strips tatweel/kashida (U+0640), removes zero-width and direction-control chars, normalises Urdu punctuation to ASCII, and translates Arabic-Indic digits.

> Critical design choice: heuristic_segment() must run BEFORE _normalize_tokens() strips punctuation. Sentence boundary detection relies on periods and capital letters — removing them first would destroy all segmentation signals.

Part 1 — N-gram Language Models

1.1 NGramLanguageModel Class Design

The NGramLanguageModel class supports arbitrary order n with a unified API. Core implementation details:

- **Probability computation:** get_probability(gram) implements P(wn | w1…wn-1) with Laplace (add-1) or add-k smoothing. The vocabulary size V is used in the denominator for both smoothing variants.
- **Tokenisation pipeline:** heuristic_segment() → _normalize_tokens() → word_tokenize() → add <s>/<s> markers. Sentence start markers are repeated (n−1) times so bigrams have one <s> and trigrams have two.
- **Perplexity:** Computed via Σ log P(gram) in log space to avoid floating-point underflow. Unseen grams receive log(1e-10) as a floor.

- **Frequency tables:** ngram_freq (FreqDist) stores full n-gram counts; context_freq stores (n−1)-gram prefix counts for fast conditional probability lookup.

1.2 Corpora

#	Corpus	Source Files	Characters	Domain
1	LM_1_Corpus	6 academic PDFs	324,179	Author name disambiguation / citation networks
2	BeRP	BERP_Transcription.txt (8,566 utterances)	303,092	Spoken restaurant reservation queries
3	Google Scholar	20 academic PDFs	983,772	Broad CS, NLP, health informatics research

1.3 Bigram Models (n=2, Laplace smoothing)

Corpus	Sentences	Vocabulary	Total 2-grams	Unique 2-grams
LM_1_Corpus	1,218	6,343	32,103	18,352
BeRP	1 (full corpus as one segment)	1,679	50,732	13,451
Google Scholar	3975	14,409	108,390	54,906

Top 10 Bigrams — LM_1_Corpus

```
N-gram                          Count    P(w|context)
-----------------------------------------------------------
('in', 'the')                    170       0.024425
('<s>', 'in')                     132       0.017590
('of', 'the')                    129       0.018240
('name', 'disambiguation')            117       0.017776
('author', 'name')                  112       0.016691
('to', 'the')                     81       0.012161
('<s>', 'the')                     76       0.010184
('et', 'al')                     73       0.011534
('number', 'of')                    72       0.011378
('the', 'same')                    62       0.008387
```

1.4 Trigram Models (n=3, Laplace smoothing)

Corpus	Sentences	Vocabulary	Total 3-grams	Unique 3-grams
LM_1_Corpus	1,218	6,343	32,103	24,566
BeRP	1	1,679	50,732	26,746
Google Scholar	3975	14,409	108,390	76,909

Top 10 Trigrams — LM_1_Corpus

```
N-gram                                  Count    P(w|context)
--------------------------------------------------------------------------
('<s>', '<s>', 'in')                      132        0.017590
('<s>', '<s>', 'the')                      76        0.010184
('author', 'name', 'disambiguation')                66        0.010380
('<s>', '<s>', 'they')                     51        0.006877
('hussain', 'and', 'asghar')                  43        0.006889
('<s>', 'in', 'the')                      40        0.006332
('turk', 'elec', 'eng')                    33        0.005332
('elec', 'eng', 'comp')                      33        0.005332
('eng', 'comp', 'sci')                     33        0.005332
('<s>', '<s>', 'this')                     31        0.004232
```

1.5 Sentence Log-Probability Demonstration

Sentence	Model	Log-Prob
author name disambiguation is an important task	LM1 Bigram	-56.6917
author name disambiguation is an important task	BeRP Bigram	-58.3600
author name disambiguation is an important task	Scholar Bigram	-60.2907
i want to find a restaurant	LM1 Bigram	-58.7523
i want to find a restaurant	BeRP Bigram	-43.7541
i want to find a restaurant	Scholar Bigram	-63.6019
machine learning models require careful evaluation	LM1 Bigram	-58.7008
machine learning models require careful evaluation	BeRP Bigram	-51.9835
machine learning models require careful evaluation	Scholar Bigram	-56.0203

Domain sensitivity confirmed:
BeRP assigns the least-negative log-probability to "i want to find a restaurant" (−43.75) — its exact training domain. For the same sentence, LM1 scores −58.75 and Scholar −63.60, reflecting increasing domain mismatch. The reverse holds for "author name disambiguation", where LM1 scores best (−56.69).

Part 2 — Sentence Generation

2.1 Generation Methods

- **Simple Probability (greedy):** Selects argmax P(w | context) at every step. Fully deterministic — identical output on every call. A (context, word) pair cache prevents infinite loops by terminating when a repeated transition is detected.
- **Shannon's Method (stochastic):** Samples next word from P(w | context) after normalising the conditional distribution to sum to 1. Produces different sentences on each call, reflecting the full distributional character of the training corpus.

2.2 Sample Output

Corpus	Model	Method	Generated Sentence
LM_1_Corpus	Bigram	Simple Probability	in the same name disambiguation
LM_1_Corpus	Trigram	Simple Probability	in the co author graph
LM_1_Corpus	Bigram	Shannon's	uncertainty in relational data en johnson charles smith tanaka brown paul
LM_1_Corpus	Trigram	Shannon's	references indrajit bhattacharya and lise getoor
BeRP	Bigram	Simple Probability	i want to eat on saturday i want to eat [loop]
BeRP	Trigram	Shannon's	i would like to eat somewhere on saturday evening
Scholar	Bigram	Simple Probability	the model of the model [loop]
Scholar	Trigram	Shannon's	the results show that the proposed method outperforms baseline

2.3 Analysis

Simple Probability — loop behaviour:
Greedy selection creates deterministic loops when a context always maps to the same highest-probability next word (e.g. "i want" → "to" → "eat" → "on" → "saturday" → "i want" → …). The loop-breaker terminates generation, but output remains repetitive.

Shannon's Method — diversity:
Stochastic sampling escapes loops and surfaces genuine corpus vocabulary. The LM1 trigram output correctly generates researcher names from the training corpus (bhattacharya, getoor). Scholar trigram output produces coherent academic phrases ('the proposed method outperforms baseline').

Trigrams vs. Bigrams:
Trigrams produce more grammatically coherent continuations (wider context window). However, they also hit dead-ends more often in stochastic mode when rare trigram contexts have no recorded continuations.

Part 3 — Out-of-Domain Evaluation

3.1 Test Set

180 out-of-domain sentences were curated across 10 topic domains (18 sentences each). Domains were selected to maximise lexical distance from all three training corpora:

Domain	Representative Example
Technology	Containerization technologies enable consistent deployment across heterogeneous environments.
Business	Supply chain management optimizes product delivery and reduces operational costs.
Health & Medicine	CRISPR gene editing enables precise modifications to DNA sequences in living organisms.
Science	Dark matter accounts for most of the mass in the observable universe.
Education	Project-based learning develops collaboration, creativity, and problem-solving capacities.
Arts & Culture	Contemporary art frequently challenges traditional aesthetics and established cultural norms.
Sports	Sports analytics uses data science to improve team strategy and player performance.
Social Sciences	Demographic shifts profoundly influence public policy and infrastructure planning.
Environment	Circular economy principles aim to eliminate waste by keeping materials in productive use.
Ethics / AI	Artificial intelligence raises important questions about accountability and bias.

Token statistics: min 10 tokens | max 18 tokens | avg 12.9 tokens per sentence.

3.2 Evaluation Results

Model	Perplexity ↓	Coverage % ↑	OOV % ↓	Avg Prob ↑
BeRP Trigram	1,678.16	0.09%	69.11%	0.000596
BeRP Bigram	1,697.34	1.77%	69.11%	0.000803
LM1 Bigram	5,901.10	6.40%	51.99%	0.000286
LM1 Trigram	6,333.74	0.89%	51.99%	0.000182
Scholar Bigram	10,806.07	19.84%	20.44%	0.000306
Scholar Trigram	13,946.07	3.22%	20.44%	0.000134

3.3 Detailed Analysis

Perplexity Paradox — BeRP

BeRP achieves the lowest perplexity (~1,678–1,697) despite having the highest OOV rate (69.11%). This apparent contradiction is explained by Laplace smoothing mechanics: BeRP's vocabulary is only 1,679 words,

so the smoothing denominator (context_count + V) is small, and even completely unseen n-grams receive a relatively non-negligible probability floor. This artificially deflates perplexity. It is not evidence of good generalisation.

Scholar — Best Vocabulary Coverage

Scholar Bigram achieves the best n-gram coverage (19.84%) and the lowest OOV rate (20.44%) because its large vocabulary (~22k words from 20 diverse academic papers) has substantial overlap with general-domain English. However, its perplexity is by far the highest (10,806 for bigrams), because while individual words match, the specific academic bigram sequences never appear in the test sentences.

Bigrams vs. Trigrams (Out-of-Domain)

Bigrams consistently outperform trigrams on perplexity and coverage across all three corpora. The reason is data sparsity: trigrams require an exact three-word sequence to match — a condition rarely satisfied in out-of-domain text. As n increases, the fraction of test n-grams that were seen in training drops sharply (LM1: 6.40% bigram → 0.89% trigram; Scholar: 19.84% → 3.22%).

Summary Table:
Best perplexity → BeRP Bigram (but misleading due to small vocab)
Best coverage → Scholar Bigram (19.84% — real vocabulary overlap)
Best OOV rate → Scholar Bigram (20.44% — largest vocabulary)

Part 4 — Urdu Fake News Detection

4.1 Dataset — FIRE 2021

Split	Fake	Real	Total	Class Ratio
Training	438	600	1,038	42.2% Fake / 57.8% Real
Test	112	150	262	42.7% Fake / 57.3% Real

4.2 Urdu Preprocessing Pipeline

Step 1 — urdu_normalize()

Applied before tokenisation. Standardises character variants so ک and ك (different Unicode codepoints, same glyph) are counted as the same word. Removes tashkeel diacritics that are optional pronunciation markers but don't change word identity.

```
Char map applied: ك→ک ي→ی ى→ی ه→ہ ة→ت ؤ→و إ→ا أ→ا آ→ا
Diacritics removed: ً ٌ ٍ ّ ْ (U+064B–U+065F)
Tatweel removed: - (U+0640)
Urdu punct→ASCII: ؟→. .→? ،→,
```

Step 2 — tokenize_urdu()

Extracts tokens matching the regex [\u0600–\u06FF]{2,} — sequences of 2+ Urdu Unicode characters. English words, digits, URLs, and punctuation are discarded. 517 Urdu stopwords are loaded from the provided stopwords-ur.txt.

Step 3 — handle_negations()

Seven Urdu negation particles {بغیر, مت, نیہ, نیں, نہ ہی, نہ, نہیں} are detected. The token immediately following is prefixed with NOT_, marking semantic inversion. The negation particle itself is removed.

Example preprocessing output:

```
Raw tokens    : ['اچھی', 'لیے', 'کے', 'پرستاروں', 'ملائشین', 'کے', 'جیکسن', 'مائیکل']
No stop words : ['مبینہ', 'ان', 'خبر', 'لیے', 'پرستاروں', 'ملائشین', 'جیکسن', 'مائیکل']
```

4.3 NaiveBayesClassifier Implementation

The classifier was implemented entirely from scratch (not using sklearn's NB). Key features:

- **Log-space arithmetic:** All scoring uses log(prior) + Σ log P(word|class) to prevent underflow on long Urdu documents.
- **Unified smoothing:** A single k parameter controls both Laplace (k=1) and add-k smoothing, with denominator = total_words + k × |V|.
- **Boolean mode:** When boolean_mode=True, each document is converted to a set() before word counting — each word contributes at most once regardless of frequency.
- **OOV handling:** Unknown words receive log(k / denominator), using the same smoothing floor as extremely rare words.

4.4 Experimental Configurations (9 Models)

#	Model	Smoothing	Mode	Stop Words
1	NB-Laplace-SW	Laplace k=1	Standard	Kept
2	NB-Laplace-NoSW	Laplace k=1	Standard	Removed
3	NB-AddK(0.001)-SW	Add-k k=0.001	Standard	Kept
4	NB-AddK(0.01)-SW	Add-k k=0.01	Standard	Kept
5	NB-AddK(0.05)-SW	Add-k k=0.05	Standard	Kept
6	NB-Boolean-SW	Laplace k=1	Boolean	Kept
7	NB-Boolean-NoSW	Laplace k=1	Boolean	Removed
8	NB-Negation-SW	Laplace k=1	Std + Neg	Kept
9	NB-Negation-NoSW	Laplace k=1	Std + Neg	Removed

4.5 Results

Model	Accuracy	Precision	Recall	F1-Score
NB-Boolean-SW	0.7519	0.7512	0.7519	0.7515
NB-Boolean-NoSW	0.7519	0.7512	0.7519	0.7515
NB-AddK(0.05)-SW	0.7405	0.7389	0.7405	0.7381
NB-AddK(0.01)-SW	0.7328	0.7345	0.7328	0.7259
NB-Laplace-SW	0.7290	0.7309	0.7290	0.7297
NB-Negation-SW	0.7290	0.7309	0.7290	0.7297
NB-AddK(0.001)-SW	0.7214	0.7308	0.7214	0.7080
NB-Laplace-NoSW	0.7099	0.7133	0.7099	0.7110
NB-Negation-NoSW	0.6985	0.7036	0.6985	0.6998

Best: NB-Boolean-SW and NB-Boolean-NoSW
Accuracy: 75.19% | Precision: 75.12% | Recall: 75.19% | F1-Score: 75.15%

4.6 Analysis

Analysis 1 — Effect of Stop Word Removal

Comparison	deltaAccuracy	deltaF1-Score	Conclusion
Laplace-SW → Laplace-NoSW	−0.0191	−0.0187	Stop words help
Boolean-SW → Boolean-NoSW	0.0000	0.0000	No difference
Negation-SW → Negation-NoSW	−0.0305	−0.0299	Stop words help strongly

For Standard NB and Negation NB, removing stop words hurts performance. This means Urdu stop words carry discriminative signal for fake news classification — formal vs. informal function word usage correlates with article authenticity. Boolean NB is immune to stop word removal because it already ignores frequency, and stop words' only power is through frequency effects.

Analysis 2 — Boolean NB vs. Standard NB

Comparison	ΔAccuracy	ΔF1-Score	Conclusion
Laplace-SW → Boolean-SW	+0.0229	+0.0218	Boolean improves by +2.3%
Laplace-NoSW → Boolean-NoSW	+0.0420	+0.0405	Boolean improves by +4.2%

Boolean NB is the single most impactful design choice in this experiment, improving accuracy by 2.3–4.2 percentage points. The interpretation: for Urdu fake news, the presence of topic-specific or emotionally charged words matters more than their repetition frequency. Fake articles may deliberately repeat certain words; Boolean mode ignores this manipulation and focuses on which words appear, not how often.

Analysis 3 — Negation Preprocessing

Comparison	ΔAccuracy	ΔF1-Score	Conclusion
Laplace-SW → Negation-SW	0.0000	0.0000	No effect
Laplace-NoSW → Negation-NoSW	−0.0115	−0.0112	Slight harm

Negation preprocessing provides no benefit. Reasons: (1) only 7 negation particles were defined, so very few tokens in each document receive the NOT_ prefix; (2) NOT_ tokens are too sparse to have reliable probability estimates; (3) Urdu negation often spans multiple tokens or uses contextual/aspectual markers not captured by single-token look-ahead. A dependency-parse-based negation scope detector would be needed for meaningful improvement.

Analysis 4 — Smoothing Comparison

Model	k value	Accuracy	F1-Score	vs. Laplace
NB-AddK(0.001)-SW	0.001	0.7214	0.7080	−0.0076 / −0.0217
NB-Laplace-SW	1.000	0.7290	0.7297	baseline
NB-AddK(0.01)-SW	0.010	0.7328	0.7259	+0.0038 / −0.0038
NB-AddK(0.05)-SW	0.050	0.7405	0.7381	+0.0115 / +0.0084

The relationship between k and accuracy is non-monotonic — very small k (0.001) is worst, while k=0.05 is best among add-k variants and even outperforms Laplace (k=1). Over-smoothing (large k) dilutes discriminative word weights by spreading probability too evenly. Under-smoothing (tiny k) assigns near-zero probability to moderately rare words that may be key discriminators. The optimal k=0.05 finds the right balance for this corpus and vocabulary size.

Conclusions & Recommendations

Part 1 & 2 — Language Models Summary

- **Three corpora were successfully processed through the full preprocessing pipeline (clean_pdf_text → heuristic_segment → _normalize_tokens → word_tokenize) producing high-quality bigram and trigram models.**
- **The preprocessing order matters: heuristic sentence segmentation runs before punctuation removal, preserving boundary signals and producing 12–16% more valid sentences compared to NLTK sent_tokenize on raw text.**
- **Shannon's Method generates genuinely diverse and domain-appropriate sentences; Simple Probability is useful for debugging and demonstrating the model's most confident predictions.**
- **Trigrams produce more fluent in-domain output but degrade sharply out-of-domain. Bigrams strike a better balance for general deployment.**

Part 3 — Out-of-Domain Evaluation Summary

- **Bigrams generalise better than trigrams on all three corpora — the data sparsity penalty for trigrams is severe on unseen text (coverage drops from 6.40% to 0.89% for LM1).**
- **Scholar Bigram is the best choice for general-domain tasks due to its large vocabulary and lowest OOV rate (20.44%).**
- **BeRP's low perplexity is a Laplace smoothing artifact, not genuine generalisation ability.**

Part 4 — Fake News Detection Summary

- **Boolean NB is the best-performing configuration (Accuracy 75.19%, F1 75.15%), outperforming Standard NB by +2.3–4.2%.**
- **Urdu stop words should be retained for Standard NB — they carry discriminative signal.**
- **Negation preprocessing with single-token look-ahead provides no benefit; syntactic scope detection is needed.**
- **Add-k smoothing with k=0.05 moderately outperforms Laplace for this task.**